

BIBLE PYTHON

Référence de révision du langage Python — syntaxe, caractères, structures et outils essentiels

Palette inspirée de VS Code · bleu = mots-clés · orange = chaînes · vert = commentaires · jaune = fonctions · vert turquoise = types · vert clair = nombres.

1. CARACTÈRES ET OPÉRATEURS

+ - * /	arithmétique
//	division entière
%	modulo / reste
**	puissance
= / == / !=	affecter / égal / différent
< > <= >=	comparaisons
+= -= *= /= //= %= **=	affectations composées
and / or / not	logique
in / not in	appartenance
is / is not	identité
& ^ ~ << >>	opérations bit à bit
() [] {}	parenthèses / listes / dictionnaires-sets
:, . # ;	bloc / séparation / attribut / commentaire / séparation

2. TYPES ET VARIABLES

int	entier
float	nombre réel
complex	complexe
bool	True / False
str	texte
list	collection ordonnée modifiable
tuple	collection ordonnée non modifiable
dict	clé → valeur
set	ensemble sans doublons
None	absence de valeur

3. CHAÎNES ET ÉCHAPPEMENTS

"texte" / 'texte'	chaînes
f"Valeur : {x}"	f-string
r"C:\dossier"	chaîne brute
\n	nouvelle ligne
\t	tabulation

<code>\\</code>	antislash
<code>\\" / \'</code>	guillemets
<code>.upper()</code> / <code>.lower()</code>	majuscules / minuscules
<code>.strip()</code>	retire les espaces aux extrémités
<code>.replace()</code>	remplacement
<code>.split()</code> / <code>.join()</code>	découper / assembler
<code>.find()</code> / <code>.count()</code>	chercher / compter
<code>.startswith()</code> / <code>.endswith()</code>	tester début / fin

4. MOTS-CLÉS PYTHON

if / elif / else	conditions
for / while	boucles
break / continue / pass	contrôle des boucles
def / return	fonctions
lambda / yield	fonction anonyme / générateur
try / except / else / finally	exceptions
raise	déclencher une exception
import / from / as	modules
with	gestionnaire de contexte
class / self / super	programmation objet
assert	vérification
global / nonlocal	portées
del	supprimer une référence / élément
match / case	pattern matching
async / await	asynchrone

5. STRUCTURES DE DONNÉES ET INDEXATION

```
liste = [10, 20, 30]
tuple = (10, 20, 30)
dictionnaire = {'$0$: $1$, $2$: 20}
ensemble = {10, 20, 30}

liste[0] # premier élément
liste[-1] # dernier
liste[1:3] # slicing
liste[:3] # début
liste[2:] # fin
liste[::2] # pas de 2
liste[::-1] # inverser
```

Méthodes principales

.append() / .insert()	ajouter / insérer
.remove() / .pop()	supprimer / retirer et renvoyer
.clear() / .copy()	vider / copier
.sort() / .reverse()	trier / inverser
.keys() / .values() / .items()	clés / valeurs / paires
.get() / .update()	récupérer / mettre à jour

6. CONDITIONS ET BOUCLES

```
if temperature > 80:
    print('$0$)
elif temperature > 50:
    print('$0$)
else:
    print('$0$)
```

```
for i in range(5):  
    print(i)  
  
while condition:  
    # instructions
```

⚠ L'indentation fait partie de la syntaxe Python.

7. FONCTIONS

```
def calculer_masse(volume, densite):  
    masse = volume * densite  
    return masse  
  
def calculer(a: float, b: float) -> float:  
    return a + b  
  
def fonction(*args, **kwargs):  
    ...
```

8. FONCTIONS INTÉGRÉES À CONNAÎTRE

<code>print()</code> / <code>input()</code>	afficher / demander une saisie
<code>len()</code> / <code>range()</code>	longueur / séquence d'entiers
<code>type()</code> / <code>isinstance()</code>	inspecter un type
<code>int()</code> / <code>float()</code> / <code>str()</code> / <code>bool()</code>	conversions
<code>abs()</code> / <code>round()</code>	valeur absolue / arrondi
<code>min()</code> / <code>max()</code> / <code>sum()</code>	min / max / somme
<code>sorted()</code> / <code>reversed()</code>	trier / inverser
<code>enumerate()</code> / <code>zip()</code>	index + valeur / associer des séquences
<code>any()</code> / <code>all()</code>	tests collectifs
<code>open()</code>	ouvrir un fichier

9. COMPRÉHENSIONS

```
carres = [x**2 for x in nombres]
pairs = [x for x in nombres if x % 2 == 0]
dico = {x: x**2 for x in nombres}
ensemble = {x for x in nombres if x > 0}
```

10. EXCEPTIONS

```
try:
    resultat = nombre / 0
except ZeroDivisionError:
    print($$0$$)
except Exception as erreur:
    print(erreur)
else:
    print($$0$$)
finally:
    print($$0$$)
```

11. MODULES ET FICHIERS

```
import math
from math import pi, sqrt
from math import sqrt as racine

with open($$0$$, $$1$$, encoding=$$2$$) as fichier:
    contenu = fichier.read()
```

12. PROGRAMMATION OBJET

```
class Piece:
    def __init__(self, nom, masse):
        self.nom = nom
        self.masse = masse

    def afficher(self):
        print(f$$0$$)

piece = Piece($$0$$, 125)
piece.afficher()
```

13. SYNTAXES INTERMÉDIAIRES

```
carres = list(map(lambda x: x**2, nombres))
grands = list(filter(lambda x: x > 10, nombres))

def fonction(*args, **kwargs):
    ...

a, *reste = [1, 2, 3]
dico = {'$0$: 1', '$1$: 2'}
copie = {'*dico', '$0$: 3'}

if __name__ == '$0$:':
    main()
```

14. FICHE EXPRESS — À SAVOIR PAR CŒUR

= / ==	affecter / comparer
+ - * // % **	calculs
and / or / not	logique
if / elif / else	conditions
for / while	boucles
break / continue	contrôle
def / return	fonctions
list / dict / tuple / set	collections
try / except	erreurs
import / from	modules
class / self	objets
[] / () / {}	structures
:	bloc / slicing
#	commentaire
.	attribut / méthode

Pour ton ICP : maîtrise d'abord parfaitement la syntaxe, les conditions, boucles, fonctions, listes/dictionnaires et fichiers. Ensuite : exceptions, modules, POO et notions avancées. Pour tes scripts de calcul de tolérances, fonctions + dictionnaires + fichiers seront particulièrement utiles.

Python 3.x · Document de révision · Couleurs inspirées de VS Code